Architektur der FECG Booking-Webseite

Gemeindefreizeit 2027 | Technische Dokumentation

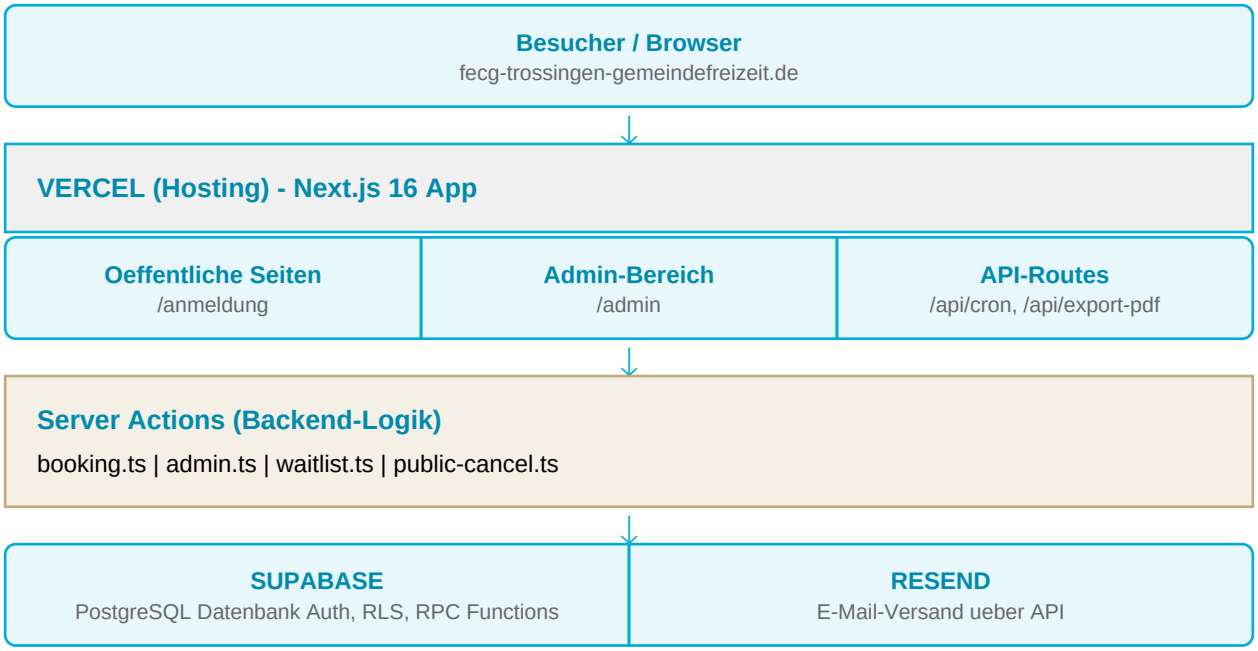
Projekt: FECG Trossingen Gemeindefreizeit 2027 - Buchungssystem
Domain: fecg-trossingen-gemeindefreizeit.de
Technologien: Next.js 16, Supabase (PostgreSQL), Vercel, Resend, Tailwind CSS
Stand: April 2026

FECG Trossingen e.V.

1. Das Gesamtbild

Die Webseite besteht aus mehreren Diensten, die zusammenarbeiten. Der Besucher sieht nur die Webseite im Browser - im Hintergrund kommunizieren aber mehrere Systeme miteinander.

Systemarchitektur



2. Die einzelnen Technologien

Next.js (das Framework)

Next.js ist das Grundgeruest der Webseite - wie das Skelett eines Hauses. Es baut auf **React** auf und hat zwei Seiten:

- **Frontend** = was der Besucher sieht (HTML, CSS, Buttons, Formulare)
- **Backend** = Server Actions, die im Hintergrund laufen (Datenbank, E-Mails)
- **Ordnerstruktur:** src/app/ = Seiten, src/components/ = Bausteine, src/lib/ = Logik

Vercel (das Hosting)

Vercel ist der Hosting-Dienst - dort laeuft die Webseite im Internet. Der Ablauf: Code wird zu GitHub gepusht, Vercel erkennt die Aenderung und baut automatisch die Webseite neu (Deployment). Vercel stellt auch die Domain bereit und verwaltet die Umgebungsvariablen (Passwoerter fuer Supabase, Resend).

Supabase (die Datenbank)

Supabase ist eine gehostete PostgreSQL-Datenbank mit Extra-Features:

- **PostgreSQL** = die eigentliche Datenbank (Tabellen, Spalten, Beziehungen)
- **RLS (Row Level Security)** = Sicherheitsregeln, wer welche Daten sehen/aendern darf
- **RPC Functions** = gespeicherte SQL-Funktionen (z.B. create_public_reservation), die als Transaktion laufen
- **Auth** = Login-System fuer den Admin-Bereich

SQL / Migrationen

SQL ist die Sprache, mit der die Datenbank gesteuert wird. Migrationen sind nummerierte SQL-Dateien (001, 002, ..., 010), die die Datenbank Schritt fuer Schritt aufbauen. Jede Migration aendert etwas: neue Tabelle, neue Spalte, neue Funktion. Diese werden im Supabase SQL-Editor ausgefuehrt.

Resend (E-Mail-Versand)

Ein Dienst, der E-Mails ueber eine API sendet. Die Webseite ruft Resend auf, Resend sendet die E-Mail an den Empfaenger. Templates sind HTML-Strings in src/lib/email/templates.ts definiert.

GitHub + Git (Versionsverwaltung)

Git speichert jede Aenderung am Code (wie eine Zeitmaschine). GitHub ist der Online-Speicherort, von dem Vercel den Code holt. Code liegt auf Netzlaufwerk Z: und wird zu GitHub gepusht. Vercel baut dann automatisch die Seite.

3. Datenbank-Struktur

Die Datenbank besteht aus mehreren Tabellen, die miteinander verknuepft sind:

Tabelle	Beschreibung	Inhalt
events	Veranstaltungen	Gemeindefreizeit 2027 mit Datum, Preisen, Bankdaten
house_types	Unterkunftstypen	Villa II, Gaestehaus G11, etc. mit Beschreibung und Kapazitaet
houses	Einzelne Haeuser	Haus 1, Haus 2, etc. - gehoeren zu einem Typ
reservations	Buchungen	Kontaktdaten, Haus, Preis, Zahlungsstatus, Token
guests	Gaeste pro Buchung	Name, Geburtsdatum, Geschlecht, Hinweise
waitlist	Warteliste	Kontaktdaten, Gaeste als JSON, Position, Status
admin_users	Admin-Zugaenge	Verknuepfung mit Supabase Auth

Beziehungen:

- events → house_types → houses → reservations → guests
- events → waitlist (mit guests_json als JSONB-Feld)

4. Ablauf einer Buchung (technisch)

Schritt	Aktion	Details
1	Formular ausfuellen	Besucher waehlt Unterkunft, traegt Kontaktdaten und Gaeste ein
2	Daten senden	Browser schickt Daten an Server Action (booking.ts)
3	Validierung	Server Action prueft Eingaben mit Zod-Schema
4	RPC aufrufen	Server ruft Supabase-Funktion create_public_reservation() auf
5	DB-Transaktion	Supabase prueft Verfuegbarkeit, erstellt Reservierung + Gaeste, gibt Token zurueck
6	E-Mails senden	Server sendet Bestaetigungs-E-Mail an Gast + Info-E-Mail an Admin ueber Resend
7	Weiterleitung	Besucher wird zur Bestaetigungsseite weitergeleitet mit Token-URL

Wichtig: Transaktionen

Die RPC-Funktion laeuft als Transaktion - entweder wird alles erstellt (Reservierung + Gaeste + Haus belegt) oder bei einem Fehler wird nichts geaendert. Das verhindert inkonsistente Daten.

5. E-Mail-System

Das System versendet automatisch E-Mails bei verschiedenen Ereignissen:

E-Mail-Template	Empfaenger	Ausloser
Reservierungsbestaetigung	An Gast	Nach erfolgreicher Buchung mit Zahlungsdaten
Admin-Benachrichtigung	An Admin	Neue Reservierung eingegangen
Warteliste-Bestaetigung	An Gast	Platz auf der Warteliste mit Position
Admin Warteliste-Info	An Admin	Neuer Warteliste-Eintrag
Buchung bestaetigt	An Gast	Zahlung eingegangen, Buchung fix
Zahlungserinnerung	An Gast	3 Tage vor Ablauf der Zahlungsfrist
Stornierungsbestaetigung	An Gast	Reservierung storniert mit Grund
Selbst-Stornierung Info	An Admin	Gast hat selbst storniert
Reservierung abgelaufen	An Gast	Zahlungsfrist ueberschritten

6. Admin-Funktionen

Funktion	Beschreibung
Zahlung bestaetigen	Setzt Status auf 'bestaetigt', sendet E-Mail an Gast
Stornieren	Mit Grund-Auswahl, E-Mail an Gast, Haus wird freigegeben
Rueckgaengig machen	Zahlung/Stornierung zuruecksetzen auf 'reserviert'
Verlaengern (+7 Tage)	Zahlungsfrist um 7 Tage verlaengern
Reservierung loeschen	Komplett aus DB entfernen, Haus freigeben
Warteliste ueberfuehren	Wartenden Eintrag in Reservierung umwandeln
PDF-Export	Zahlungsuebersicht als druckbare PDF-Seite
Admin-Notizen	Interne Notizen pro Reservierung

7. Wichtige Dateien im Projekt

Pfad	Beschreibung
src/app/	Alle Seiten (Routing ueber Ordnerstruktur)
src/components/	Wiederverwendbare UI-Bausteine
src/lib/actions/	Server Actions (Backend-Logik)
src/lib/email/	E-Mail-Templates und Versand
src/lib/supabase/	Datenbankverbindung (Client + Admin)
src/lib/validations/	Eingabe-Validierung (Zod-Schemas)
supabase/migrations/	SQL-Dateien fuer die Datenbankstruktur
.env.local	Geheime Schluessel (nur lokal, nicht in Git)
src/app/globals.css	Globale Styles, Farb-Variablen

8. Neue Webseite erstellen - Schritt fuer Schritt

Phase 1: Projekt aufsetzen

1. **Node.js installieren** - die Laufzeitumgebung fuer JavaScript (nodejs.org)
2. **Next.js Projekt erstellen** - npx create-next-app@latest
3. **GitHub Repository erstellen** - fuer die Versionsverwaltung
4. **Vercel Konto erstellen** und mit GitHub verbinden

Phase 2: Datenbank einrichten

5. **Supabase Projekt erstellen** auf supabase.com (kostenlos)
6. **Tabellen entwerfen** - welche Daten brauchst du?
7. **SQL-Migrationen schreiben** - Tabellen, Spalten, Beziehungen
8. **RLS-Policies erstellen** - Sicherheitsregeln (wer darf was?)
9. **RPC-Funktionen** - fuer komplexe Operationen als Transaktion

Phase 3: Backend bauen

10. **Supabase Client einrichten** - Verbindung zur Datenbank
11. **Server Actions schreiben** - die Geschaeftslogik
12. **Validierung einbauen** - Eingaben pruefen (z.B. mit Zod)
13. **E-Mail-Integration** - Resend-Konto, Templates, Versand

Phase 4: Frontend bauen

14. **Seiten erstellen** - Layout, Navigation, Formulare
15. **Komponenten bauen** - wiederverwendbare UI-Teile
16. **Styling mit Tailwind CSS** - Farben, Abstaende, Responsive Design
17. **shadcn/ui nutzen** - fertige UI-Komponenten (Dialog, Badge, Table, ...)

Phase 5: Admin-Bereich + Deployment

18. **Login mit Supabase Auth** einrichten
19. **Dashboard + Verwaltungsseiten** bauen
20. **Umgebungsvariablen** in Vercel setzen
21. **Domain verbinden** + DNS-Eintraege setzen
22. **Testen** - alles durchklicken, E-Mails pruefen

9. Monatliche Kosten

Dienst	Beschreibung	Kosten
Vercel	Hosting	Kostenlos (Hobby-Plan)
Supabase	Datenbank + Auth	Kostenlos (Free Tier: 500MB DB)
Resend	E-Mail-Versand	Kostenlos (bis 100 E-Mails/Tag)

Dienst	Beschreibung	Kosten
Domain	fecg-trossingen-gemeindefreizeit.de	ca. 10-15 Euro/Jahr
Gesamt		ca. 1 Euro/Monat (nur Domain)

Hinweis

Alle verwendeten Dienste bieten grosszuegige kostenlose Tarife, die fuer dieses Projekt voellig ausreichen. Erst bei sehr hohem Traffic oder grossen Datenmengen wuerden kostenpflichtige Tarife noetig.